

Project Documentation: Automated ETL Pipeline for JSONPlaceholder Data

Version: 1.0

Date: September 25, 2025

Objective of the Project

This project implements an automated Extract, Transform, Load (ETL) pipeline designed to ingest data from the JSONPlaceholder public API. It processes various data entities such as users, posts, comments, albums, photos, and todos, transforming the nested JSON structures into a normalized relational format. The processed data is then loaded into a MySQL database, with a continuous update mechanism to ensure data synchronization. This pipeline provides a robust solution for maintaining an up-to-date local replica of the API data, facilitating data analysis and reporting.

Table of Contents

1. [Project Objective](#)
 2. [ETL Architecture](#)
 - [Extract](#)
 - [Transform](#)
 - [Load](#)
 3. [Workflow](#)
 4. [Database Schema](#)
 5. [Tools Used](#)
 6. [Challenges Faced](#)
-

1. Project Objective

The primary objective of this project is to create a robust and automated ETL (Extract, Transform, Load) pipeline. This pipeline fetches data from a public API (jsonplaceholder.typicode.com), processes it, and stores it in a structured relational database (MySQL). The system is designed to run continuously, periodically checking for new data and updating the database accordingly, ensuring data synchronization between the source and the destination.

2. Architecture of the ETL

The project follows a classic ETL architecture, which is broken down into three main stages:

Extract

- **Data Source:** The script extracts data from the JSONPlaceholder REST API, which provides mock JSON data through various endpoints (e.g., `/users`, `/posts`, `/todos`).
- **Extraction Mechanism:** The `requests.get()` method is used to send HTTP GET requests to these API endpoints. The `.json()` method then parses the JSON response into a Python dictionary, making it ready for the transformation stage.

Transform

- **Data Normalization:** The raw JSON data, which can be nested, is flattened into a tabular structure using `pandas.json_normalize()`. This is a crucial step as it converts semi-structured JSON into a 2D DataFrame, suitable for relational databases. For example, nested JSON objects like `address` are flattened into separate columns (e.g., `address_street`, `address_city`).
- **Schema Inference:** The script dynamically infers the data types of each column from the pandas DataFrame (e.g., `int64`, `object`, `bool`). It then translates these pandas data types into corresponding MySQL data types (`int`, `varchar(600)`, `tinyint`) to automatically generate the `CREATE TABLE` SQL statements.
- **Data Formatting:** Before insertion, the script handles data cleaning by converting boolean values to integers (`1` or `0`) and managing `NaN` (Not a Number) values by converting them to SQL `NULL`.

Load

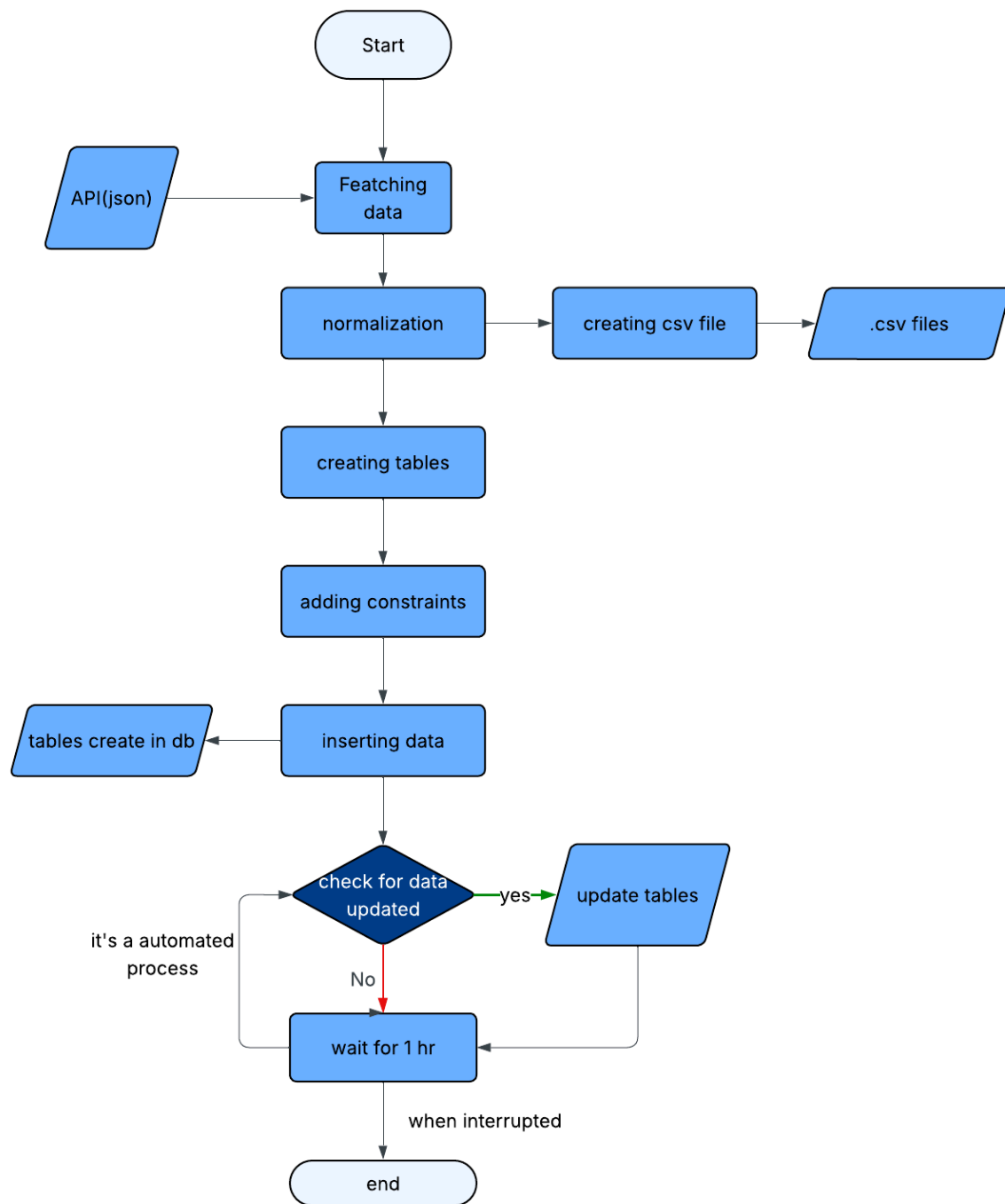
- **Database and Table Creation:** The script programmatically creates a MySQL database named `mydb2` if it doesn't already exist. It then executes the dynamically generated `CREATE TABLE` statements to build the required tables (`users`, `posts`, etc.).
- **Data Insertion:** The transformed data is loaded into the corresponding MySQL tables. The script uses the `executemany()` method for efficient bulk insertion of records. To

prevent crashes from duplicate entries during repeated runs, it uses an `INSERT IGNORE` statement. This command tells MySQL to simply discard any new rows that would cause a duplicate key violation, making the process idempotent.

- **Establishing Relationships:** After creating the tables, the script adds primary keys to each table and foreign key constraints to link related tables (e.g., linking `posts.userId` to `users.id`). This enforces relational integrity within the database.
-

3. Workflow

The script executes in a logical sequence, as illustrated in the workflow diagram.

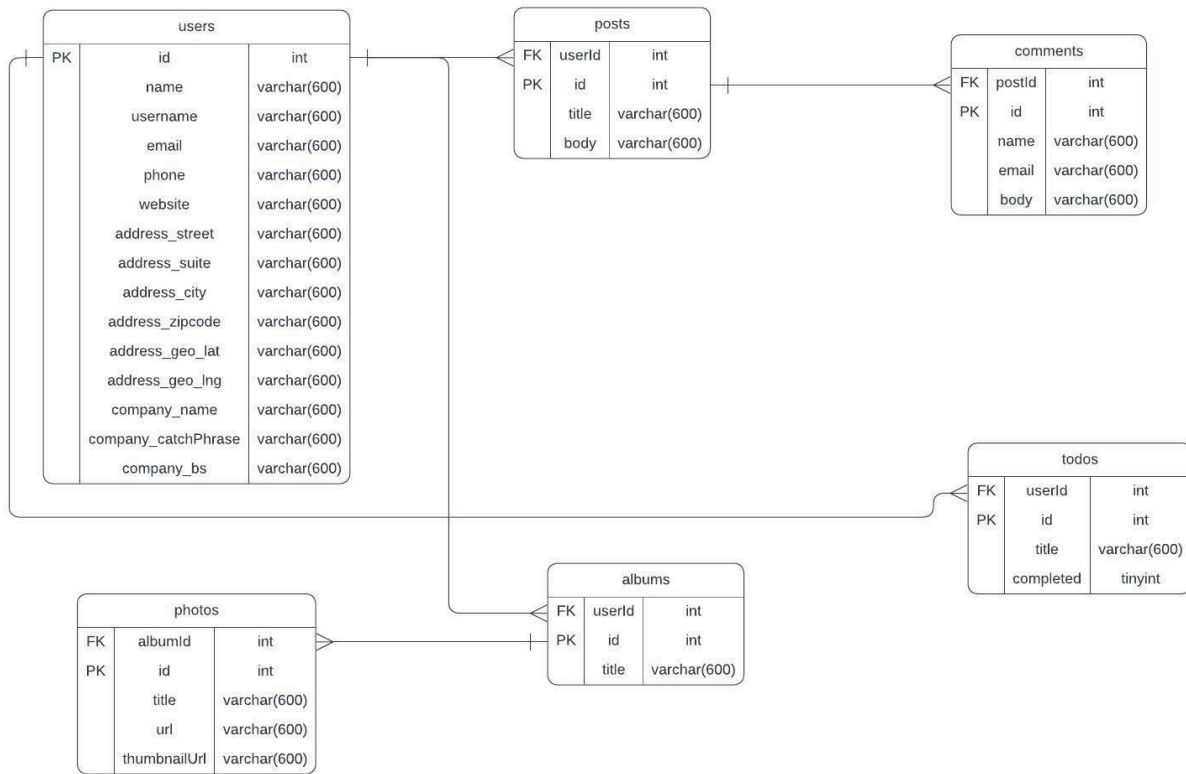


Flow diagram

1. **Initialization:** The script defines a list of API endpoints (`users`, `posts`, `comments`, etc.) to be processed.
 2. **Table Creation:** It iterates through each endpoint URL. For each one, it:
 - Fetches the JSON data.
 - Normalizes the data into a pandas DataFrame.
 - Saves a backup of the raw data as a `.csv` file.
 - Infers the schema and dynamically constructs and executes a `CREATE TABLE` SQL query.
 3. **Add Constraints:** Once all tables are created, the script executes `ALTER TABLE` commands to define primary keys for each table and establish foreign key relationships between them.
 4. **Initial Data Load:** The script loops through the URLs again, this time inserting the fetched data into the newly created tables.
 5. **Automated Update Cycle:** After the initial setup and data load, the script enters an infinite `while(True)` loop. In this loop, it:
 - Iterates through each URL and attempts to insert data using `INSERT IGNORE`.
 - Pauses execution for one hour (`time.sleep(60*60)`).
 - This cycle repeats indefinitely to capture new data.
 6. **Error Logging:** Any exceptions are caught and logged with a timestamp to an `error.txt` file for debugging.
-

4. Database Schema

The ETL process results in the following relational database schema, which normalizes the data from the JSONPlaceholder API into six distinct tables.



Database Schema

- **users**: Stores user information. It is the central table, linked to several other tables.
 - **posts**: Contains posts made by users. It has a many-to-one relationship with **users**.
 - **comments**: Holds comments on posts. It has a many-to-one relationship with **posts**.
 - **albums**: Stores photo albums created by users. It has a many-to-one relationship with **users**.
 - **photos**: Contains individual photos. It has a many-to-one relationship with **albums**.
 - **todos**: Stores to-do list items for users. It has a many-to-one relationship with **users**.
-

5. Tools Used in This Project

Tool/Library	Purpose
<code>requests</code>	Used to perform HTTP GET requests to fetch data from the JSONPlaceholder API.
<code>pandas</code>	A critical library used for data manipulation and transformation, especially for flattening JSON.
<code>mysql.connector</code>	The official MySQL driver for Python, used to connect to the MySQL server and execute SQL queries.
<code>datetime</code>	Used to generate timestamps for logging errors.
<code>time</code>	Used to pause the script's execution in the continuous update loop.

6. Challenges Faced

Developing this ETL pipeline involved overcoming several common challenges:

- **Handling Nested JSON:** The source data is in a nested JSON format. This was effectively solved using `pandas.json_normalize()`.
- **Dynamic Schema Generation:** Rather than hardcoding the table structure, the script dynamically creates it by inferring data types. This makes the pipeline more flexible but requires careful mapping between pandas and SQL data types.
- **Data Idempotency:** Since the script runs continuously, it must handle repeated data insertions without causing errors or creating duplicates. This was addressed by using `INSERT IGNORE`, which makes the loading process idempotent.
- **Error Handling and Robustness:** Network issues or database connection problems can interrupt the pipeline. The use of `try...except` blocks and logging to `error.txt` makes the system more robust by preventing crashes and providing a record of failures.
- **Maintaining Relational Integrity:** Ensuring that primary and foreign keys are correctly set up is crucial. The script separates table creation from constraint addition and ignores errors if constraints already exist, contributing to its robustness.